



Pasión por la Tecnología

Fundamentos de Gestión de Datos

Docente:
Pilar Rocío Sayán Mejía

LABORATORIO CALIFICADO N° 02

Semana 3 y 4:

*Normalización de Bases de Datos y
Consultas SQL Básicas*



LABORATORIO CALIFICADO N° 02 Tema: Normalización de Bases de Datos y Consultas SQL Básicas	
DOCENTE: Pilar Rocío Sayán Mejía	CURSO: Fundamentos de Gestión de Datos

Número de grupo	Sección
Integrante 1	
Integrante 2	
Integrante 3	
Integrante 4	
Fecha	

I. Elemento de la Capacidad Terminal de las Semanas 3 y 4

Aplica consultas SQL avanzadas con INNER JOIN, LEFT JOIN, GROUP BY y HAVING sobre una base de datos real, e identifica las formas normales (1FN, 2FN, 3FN) construyendo el modelo Entidad-Relación del minimarket a partir de la exploración directa de sus tablas, comunicando los hallazgos como equipo de analistas de datos.

II. Seguridad

En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red sin supervisión.

Prohibida la ingesta de alimentos y bebidas dentro del laboratorio.

Ubicar maletines y mochilas en el lugar destinado para tal fin.

Dejar la mesa de trabajo y la silla limpias y ordenadas al finalizar.

Todo el código se ejecuta únicamente en Google Colab mediante SQLite. No se requiere instalar software adicional.

III. Fundamento Teórico

Revise el material de las semanas correspondientes antes de iniciar el laboratorio:

- Semana 3 — Diapositivas: SQL Avanzado: INNER JOIN, LEFT JOIN, GROUP BY, HAVING, subconsultas básicas.
- Semana 4 — Diapositivas: Modelo Entidad-Relación, Cardinalidad y Normalización (1FN, 2FN, 3FN).

Conceptos clave: INNER JOIN, LEFT JOIN, GROUP BY, HAVING, WHERE vs HAVING, clave primaria (PK), clave foránea (FK), cardinalidad 1:N, redundancia de datos, anomalías de actualización, 1FN, 2FN, 3FN, dependencia funcional, dependencia parcial, dependencia transitiva, modelo Entidad-Relación (ER), SQLite, pandas, pd.read_sql_query.

IV. Normas Empleadas

No aplica norma técnica específica para este laboratorio.

V. Recursos

- Computadora con acceso a internet y navegador web.
- Cuenta de Google para acceder a Google Colab.
- Base de datos: minimarket_normalizacion.db — se descarga automáticamente desde GitHub.
- Librerías: sqlite3 (estándar Python) y pandas (preinstalada en Colab).
- URL de la base de datos:
https://raw.githubusercontent.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos/main/casos/18_minimarket_normalizacion/minimarket_normalizacion.db

VI. Metodología para el Desarrollo de la Tarea

El laboratorio se desarrolla de forma grupal (máximo 4 integrantes) en un tiempo estimado de 2 horas.

Todos los integrantes del grupo deben participar activamente en las respuestas.

Entregable: Notebook .ipynb ejecutado con código, resultados visibles y respuestas completas a todas las preguntas.

Subir el enlace del notebook al foro de la semana según indicación de la docente.

VII. Procedimiento

Caso de Negocio: MiniMarket Normalizado

Ustedes son un equipo de analistas de datos contratados por un minimarket de la ciudad. La empresa cuenta con una base de datos que fue normalizada recientemente: antes, toda la información de ventas se registraba en una sola tabla (ventas_original) donde los datos de clientes, productos, categorías y locales se repetían en cada fila. Ahora la base de datos está organizada en tablas separadas. El objetivo del equipo es: (1) explorar la estructura y construir el modelo ER, (2) aplicar consultas SQL avanzadas

para responder preguntas del negocio, (3) identificar qué formas normales se aplicaron y por qué, y (4) comunicar los hallazgos con claridad. Importante: cada consulta va acompañada de una pregunta de interpretación. Las respuestas deben conectar el resultado con una decisión real del negocio.

Actividad 1: Revisión de Conceptos

Completen la siguiente tabla con sus propias palabras. No copien textualmente del material. Usen el caso del minimarket para los ejemplos de las columnas 3 y 4.

Nº	Concepto	Definición con sus palabras	Ejemplo del minimarket
1	INNER JOIN		
2	LEFT JOIN		
3	GROUP BY		
4	HAVING		
5	Diferencia entre WHERE y HAVING		
6	Clave Primaria (PK)		
7	Clave Foránea (FK)		
8	Primera Forma Normal (1FN)		
9	Segunda Forma Normal (2FN)		
10	Tercera Forma Normal (3FN)		

Actividad 2: Exploración de la Base de Datos y Modelo ER

Antes de responder las consultas de negocio, el equipo debe comprender cómo está organizada la base de datos y cómo se relacionan sus tablas. Ejecuten cada celda en orden y analicen la estructura relacional del minimarket a partir del modelo Entidad–Relación (ER) proporcionado.

Esta actividad busca que interpreten la lógica del diseño relacional y comprendan cómo la normalización ayuda a evitar redundancia e inconsistencias en los datos.

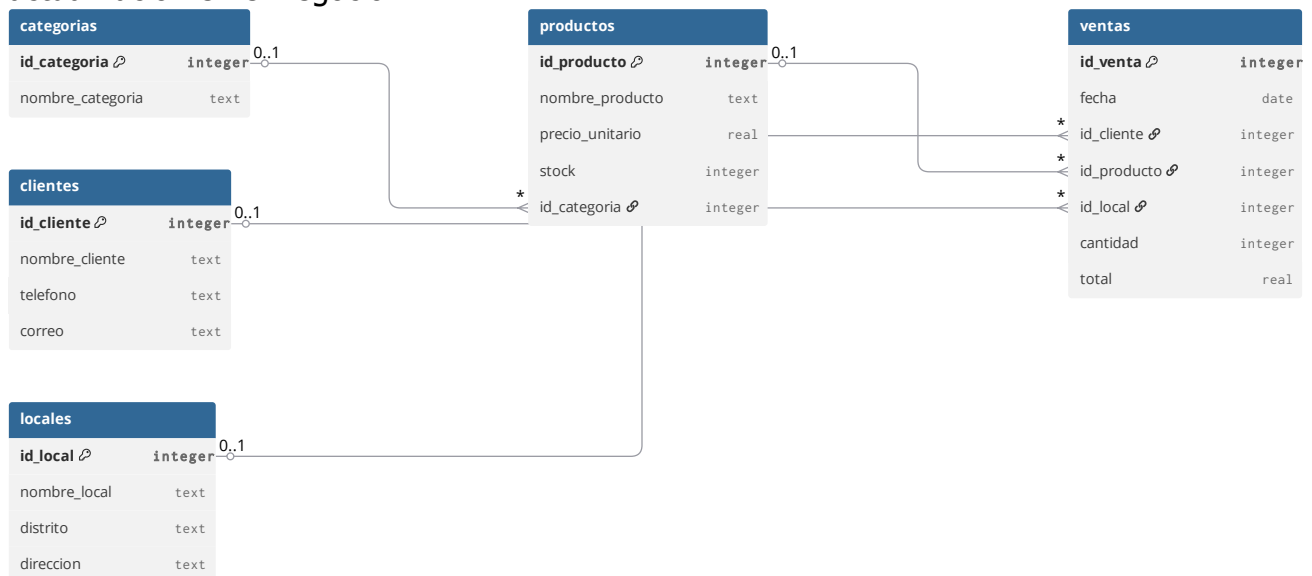
Pregunta de interpretación

Interpreten el siguiente modelo Entidad–Relación (ER) del minimarket:

<https://dbdiagram.io/d/6a15efbab62396d22c76dd82>

Expliquen:

- cómo se relacionan las tablas entre sí,
- cuál es la función de las claves primarias (PK) y claves foráneas (FK),
- y por qué este diseño normalizado ayuda a evitar redundancia de datos y problemas de actualización en el negocio.



Paso 0 — Descargar la base de datos desde GitHub y conectarse

Ejecuten esta celda primero. Descarga la base de datos del minimarket desde el repositorio GitHub. Línea por línea: `curl` descarga el archivo `.db` desde la URL de GitHub; `sqlite3.connect` abre la conexión; `pd.read_sql_query` ejecuta consultas y devuelve tablas `pandas`.

```

import sqlite3
import pandas as pd
import subprocess
from pathlib import Path

url = (
    'https://raw.githubusercontent.com/Rociosayan/'
    'PMD1_Fundamentos_Gestion_Datos/main/casos/'
    '18_minimarket_normalizacion/minimarket_normalizacion.db'
)
db_path = Path('minimarket_normalizacion.db')

# Descargar la base de datos desde GitHub.
# Se usa curl para evitar problemas de certificados SSL en algunas
instalaciones locales de Python.
  
```

```

comando = ['curl', '--ssl-no-revoke', '-L', '--fail', '-o', str(db_path), url]
resultado = subprocess.run(comando, capture_output=True, text=True)

# En algunos entornos Linux/Colab, curl puede no necesitar --ssl-no-revoke.
if resultado.returncode != 0:
    comando = ['curl', '-L', '--fail', '-o', str(db_path), url]
    resultado = subprocess.run(comando, capture_output=True, text=True)

if resultado.returncode != 0:
    raise RuntimeError('No se pudo descargar la base de datos desde GitHub.
Revisa tu conexi?n a Internet.')

if not db_path.exists() or db_path.stat().st_size == 0:
    raise RuntimeError('La base de datos se descarg? vac?a. Vuelve a ejecutar
la celda.')

conn = sqlite3.connect(db_path)
print('Base de datos del minimarket cargada correctamente.')

```

Paso 1 — Explorar las tablas disponibles

Observen qué tablas existen en la base de datos. Cada tabla representa una entidad del negocio.

```

# Ver todas las tablas de la base de datos
df_tablas = pd.read_sql_query(
    '''
        SELECT name AS tabla
        FROM sqlite_schema
        WHERE type = 'table'
        AND name NOT LIKE 'sqlite_%'
        ORDER BY name
    ''',
    conn
)
display(df_tablas)

```

Paso 2 — Estructura de cada tabla

Explore las columnas y claves de cada tabla. Esta información es la base para construir el modelo ER. Observen qué columnas terminan en "_id" o "id_" — esas son claves foráneas (FK).

```

# Columnas de la tabla categorias
df_cat_info = pd.read_sql_query('SELECT * FROM categorias', conn)
display(df_cat_info)

```

```

# Columnas de la tabla productos
df_prod_info = pd.read_sql_query('SELECT * FROM productos', conn)
display(df_prod_info)

```

```
# Columnas de la tabla clientes
df_cli_info = pd.read_sql_query('SELECT * FROM clientes', conn)
display(df_cli_info)
```

```
# Columnas de la tabla locales (o tiendas)
df_loc_info = pd.read_sql_query('SELECT * FROM locales', conn)
display(df_loc_info)
```

```
# Primeros registros de la tabla ventas normalizada
df_ven_info = pd.read_sql_query('SELECT * FROM ventas LIMIT 10', conn)
display(df_ven_info)
```

Construyan el Modelo ER del Minimarket

Basándose en lo que observaron en el Paso 2, completen la siguiente tabla. Esta tabla demuestra si el equipo comprendió la estructura relacional de la base de datos.

Entidad (tabla)	Atributos principales	Clave Primaria (PK)	Claves Foráneas (FK)	Relación con otras tablas
categorias				
productos				
clientes				
locales				
ventas				

Pregunta 1:

¿Cuántas claves foráneas tiene la tabla ventas? ¿A qué tablas apunta cada una? ¿Por qué ventas es la tabla central del modelo?

Respuesta:

Pregunta 2:

¿Qué cardinalidad existe entre clientes y ventas? ¿Un cliente puede tener varias ventas o una venta puede pertenecer a varios clientes? Justifiquen con el modelo construido.

Respuesta:

Pregunta 3:

¿Por qué la tabla productos usa id_categoria (un número) en lugar de guardar directamente el nombre de la categoría? ¿Qué forma normal justifica esta decisión?

Respuesta:

Actividad 3: SQL Avanzado — Semana 3

Aplicuen consultas SQL avanzadas para responder preguntas reales del negocio. Después de cada resultado, respondan la pregunta de interpretación.

Paso 3 — INNER JOIN: Reporte completo de ventas

El gerente necesita un reporte que muestre, en una sola tabla, el nombre del cliente, el producto comprado, el local y el monto de cada venta. Se necesitan tres INNER JOIN para unir las cuatro tablas mediante sus claves foráneas.

```
# Reporte completo de ventas con INNER JOIN
df_reporte = pd.read_sql_query(
    '''
    SELECT
        v.id_venta,
        v.fecha,
        c.nombre_cliente,
        p.nombre_producto,
        l.nombre_local,
        v.cantidad,
        p.precio_unitario,
        ROUND(v.cantidad * p.precio_unitario, 2) AS total_venta
    FROM ventas v
    INNER JOIN clientes c ON v.id_cliente = c.id_cliente
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN locales l ON v.id_local = l.id_local
    ORDER BY v.fecha DESC
    LIMIT 15
    ''',
    conn
)
display(df_reporte)
```

Pregunta 4:

¿Por qué fue necesario usar tres INNER JOIN para generar este reporte? ¿Qué información NO aparecería si solo consultaran la tabla ventas sin los JOIN?

Respuesta:

Pregunta 5:

Un INNER JOIN solo muestra ventas donde el cliente, el producto y el local existen en sus respectivas tablas. ¿Qué pasaría si una venta tuviera un id_cliente que no existe en la tabla clientes? ¿Aparecería en el resultado?

Respuesta:

Paso 4 — LEFT JOIN: Clientes sin compras

El área de marketing quiere identificar a los clientes registrados que nunca han realizado una compra, para enviarles una campaña de activación. El LEFT JOIN muestra TODOS los clientes aunque no tengan ventas.

```
# Todos los clientes y cuantas compras ha realizado cada uno
df_left = pd.read_sql_query(
    '''
    SELECT
        c.nombre_cliente,
        COUNT(v.id_venta) AS total_compras
    FROM clientes c
    LEFT JOIN ventas v ON c.id_cliente = v.id_cliente
    GROUP BY c.id_cliente
    ORDER BY total_compras ASC
    ''',
    conn
)
display(df_left)
```

Pregunta 6:

¿Cuántos clientes no han realizado ninguna compra (total_compras = 0)? ¿Por qué el INNER JOIN del Paso 3 NO permite detectarlos? ¿En qué se diferencia el LEFT JOIN del INNER JOIN en este caso?

Respuesta:

Paso 5 — GROUP BY: Ventas por categoría

La administradora quiere saber cuánto vende el minimarket por categoría de producto para planificar el abastecimiento del mes. Se usan dos JOIN para conectar ventas → productos → categorías.

```
# Total de ventas e ingresos por categoría
df_cat = pd.read_sql_query(
    '''
    SELECT
        cat.nombre_categoria,
        COUNT(v.id_venta) AS cantidad_ventas,
        ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS ingresos_totales
    FROM ventas v
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN categorias cat ON p.id_categoria = cat.id_categoria
    GROUP BY cat.id_categoria, cat.nombre_categoria
    ORDER BY ingresos_totales DESC
    ''',
    conn
)
display(df_cat)
```

Pregunta 7:

¿Qué categoría genera más ingresos para el minimarket? ¿Qué decisión de abastecimiento le recomendarían al administrador basándose en este resultado?

Respuesta:

Paso 6 — HAVING: Locales con ventas significativas

El gerente quiere analizar únicamente los locales con más de 20 ventas registradas. HAVING filtra grupos después del GROUP BY — no puede usarse WHERE porque cantidad_ventas es un resultado del agrupamiento, no un campo directo de la tabla.

```
# Locales con mas de 3 ventas registradas
df_having = pd.read_sql_query(
    '''
    SELECT
        l.nombre_local,
        l.districto_local,
        COUNT(v.id_venta) AS cantidad_ventas,
        ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS ingresos_totales
    FROM ventas v
    INNER JOIN locales l ON v.id_local = l.id_local
    INNER JOIN productos p ON v.id_producto = p.id_producto
    GROUP BY l.id_local, l.nombre_local, l.districto_local
    HAVING cantidad_ventas > 3
    ORDER BY ingresos_totales DESC
    ''',
    conn
)
display(df_having)
```

Pregunta 8:

¿Cuántos locales superan las 3 ventas? ¿Por qué se usó HAVING en lugar de WHERE para filtrar por cantidad de ventas? ¿Podría haberse usado WHERE cantidad_ventas > 20 directamente? Justifiquen.

Respuesta:

Paso 7 — Análisis integrador: TOP 5 productos más rentables

El dueño quiere saber cuáles son los 5 productos que generan más ingresos, considerando la cantidad vendida y el precio. Con esa información decidirá qué productos potenciar.

```
# Top 5 productos por ingresos totales
df_top = pd.read_sql_query(
    '''
    SELECT
        p.nombre_producto,
        cat.nombre_categoria,
        COUNT(v.id_venta) AS veces_vendido,
        SUM(v.cantidad) AS unidades_totales,
        ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS ingresos_totales
    FROM ventas v
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN categorias cat ON p.id_categoria = cat.id_categoria
    GROUP BY p.id_producto, p.nombre_producto, cat.nombre_categoria
    ORDER BY ingresos_totales DESC
    LIMIT 5
    ''',
    conn
)
display(df_top)
```

```
'''
conn
)
display(df_top)
```

Pregunta 9:

¿Cuál es el producto más rentable del minimarket? ¿A qué categoría pertenece? ¿Coincide con el producto más vendido en unidades? ¿Qué conclusión pueden sacar sobre precio vs. volumen de ventas?

Respuesta:

Actividad 4: Normalización — Semana 4

Analicen la tabla original desnormalizada del minimarket y compárenla con el modelo normalizado que exploraron. El objetivo es identificar qué formas normales se aplicaron y por qué.

Paso 8 — La tabla original: ventas_original

Antes de la normalización, el minimarket registraba todo en una sola tabla. Observen su estructura: los mismos datos de cliente, producto y categoría se repiten en cada fila.

```
# Ver la tabla original desnormalizada
df_orig = pd.read_sql_query(
    'SELECT * FROM ventas_original LIMIT 15',
    conn
)
display(df_orig)
```

Pregunta 10:

¿Qué columnas de ventas_original se repiten en varias filas? Mencionen al menos tres columnas con datos repetidos y expliquen qué problema concreto genera cada una para el minimarket.

Respuesta:

Paso 9 — ¿Cuántas veces se repite la información?

Cuantifiquen el problema de redundancia: ¿cuántas veces aparece cada producto en la tabla original? Esto muestra cuántos registros habría que actualizar si cambia un dato.

```
# Cuántas veces se repite cada producto en ventas_original
df_rep = pd.read_sql_query(
    '''
    SELECT producto, COUNT(*) AS veces_repetido
```

```

FROM ventas_original
GROUP BY producto
ORDER BY veces_repetido DESC
LIMIT 10
'''
conn
)
display(df_rep)

```

Pregunta 11:

¿Cuántas veces aparece el producto más repetido en ventas_original? Si el precio de ese producto cambiara, ¿en cuántos registros habría que hacer la corrección manualmente? ¿Cómo soluciona este problema el diseño normalizado de la Actividad 2?

Respuesta:

Paso 10 — Comparación: tabla original vs. modelo normalizado

Comparen directamente la misma venta en la tabla original y en la tabla normalizada. El JOIN reconstruye la misma información usando tablas separadas.

```

# La misma informacion en la tabla normalizada (con JOIN)
df_norm = pd.read_sql_query(
    '''
    SELECT
        v.id_venta,
        c.nombre_cliente,
        p.nombre_producto,
        cat.nombre_categoria,
        l.nombre_local,
        v.cantidad,
        p.precio_unitario,
        ROUND(v.cantidad * p.precio_unitario, 2) AS total_venta
    FROM ventas v
    INNER JOIN clientes c ON v.id_cliente = c.id_cliente
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN categorias cat ON p.id_categoria = cat.id_categoria
    INNER JOIN locales l ON v.id_local = l.id_local
    LIMIT 15
    '''
    conn
)
display(df_norm)

```

Pregunta 12:

Comparen ventas_original con el resultado del JOIN. ¿En qué se diferencian? ¿Por qué el resultado del JOIN muestra la misma información que la tabla original, pero el diseño de la base de datos es completamente diferente?

Respuesta:

Pregunta 13 — Formas normales:

Para cada forma normal, indiquen qué tabla del minimarket la cumple y qué problema concreto resuelve:

Forma Normal	Tabla que la cumple	Problema que resuelve en el minimarket
1FN		
2FN		
3FN		

Análisis Reflexivo Grupal

Respondan estas preguntas como equipo. No hay código en esta sección — solo razonamiento. Conecten la teoría con decisiones reales de diseño de bases de datos.

Pregunta 14:

El minimarket quiere agregar el teléfono del proveedor de cada producto. Un desarrollador propone agregar una columna telefono_proveedor directamente en la tabla productos. ¿Qué forma normal se violaría con esta decisión? ¿Por qué? ¿Qué harían ustedes como equipo de analistas?

Respuesta:

Pregunta 15:

Un cliente del minimarket quiere comprar varios productos distintos en una sola venta (como en un supermercado real). ¿La estructura actual de la tabla ventas lo permite? Si no, ¿qué tabla nueva habría que crear y qué columnas tendría?

Respuesta:

Conclusiones Grupales

Redacten cinco conclusiones como equipo sobre lo aprendido en este laboratorio. Cada conclusión debe conectar la teoría con el caso real del minimarket:

1. Sobre INNER JOIN y LEFT JOIN: ¿cuándo usaría cada uno el equipo en el minimarket?

R: _____

2. Sobre GROUP BY y HAVING: ¿qué diferencia práctica observaron al analizar ventas por categoría y por local?

R: _____

3. Sobre el modelo ER construido: ¿qué fue lo más difícil de identificar? ¿Qué aprendieron construyéndolo?

R: _____

4. Sobre las formas normales: ¿qué problema concreto del minimarket resuelve cada una (1FN, 2FN, 3FN)?

R: _____

5. Conclusión integradora: ¿por qué un minimarket real necesita una base de datos normalizada con JOINS en lugar de una sola tabla Excel?

R: _____

Material Complementario

- Coronel, C. & Morris, S. (2019). Database Systems: Design, Implementation & Management. Cengage. Capítulos 4 y 5.
- Documentación oficial SQLite: <https://www.sqlite.org/docs.html>
- Repositorio del curso — base de datos minimarket:
https://github.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos
- Diapositivas Semana 3: SQL Avanzado (JOIN, GROUP BY, HAVING) — disponibles en el aula virtual.
- Diapositivas Semana 4: Modelo Entidad-Relación, cardinalidad y normalización — disponibles en el aula virtual.

Rúbrica Holística de Evaluación

Criterio de evaluación	Excelente	Bueno	Requiere mejora	No aceptable	Puntaje logrado
Revisión de conceptos — Actividad 1 /3 pts	Completa los 10 conceptos con definiciones propias y ejemplos precisos del minimarket.	Completa la mayoría con definiciones aceptables.	Completa parcialmente o copia textualmente.	No completa la tabla de conceptos.	
Interpretación del modelo ER — Actividad 2/3 pts	Interpreta correctamente el modelo ER: identifica entidades, atributos, PK, FK, cardinalidades y explica con claridad cómo las relaciones ayudan a evitar redundancia e inconsistencias en la base de datos.	Interpreta la mayoría de entidades y relaciones del modelo ER, aunque presenta pequeñas imprecisiones en cardinalidades, PK/FK o en la explicación de la normalización.	Reconoce parcialmente las entidades o relaciones, pero presenta errores importantes en la interpretación del modelo relacional o en la explicación de la normalización.	No interpreta correctamente el modelo ER o las respuestas no guardan relación con la estructura relacional de la base de datos.	
SQL Avanzado (INNER JOIN, LEFT JOIN, GROUP BY, HAVING) — Actividad 3 /4 pts	Ejecuta todas las consultas, los resultados son correctos y las respuestas interpretan el negocio con claridad.	Ejecuta la mayoría de consultas con resultados correctos, pero interpretación incompleta.	Ejecuta las consultas, pero no interpreta los resultados o la interpretación es incorrecta.	No ejecuta las consultas o los resultados son incorrectos.	
Análisis de normalización (formas normales) — Actividad 4 /4 pts	Identifica columnas repetidas, cuantifica el problema, compara modelos y completa correctamente la tabla de formas normales.	Identifica la mayoría de problemas de normalización con análisis aceptable.	Solo identifica parcialmente los problemas o la tabla de formas normales está incompleta.	No analiza la normalización.	
Análisis reflexivo grupal y	Responde ambas	Responde ambas	Responde solo una pregunta o	No responde o las	

Criterio de evaluación	Excelente	Bueno	Requiere mejora	No aceptable	Puntaje logrado
conclusiones— 2 pts	preguntas con razonamiento sólido, conectando teoría con decisiones de diseño reales. Cinco conclusiones propias, concretas y conectadas con el caso real del minimarket.	preguntas con razonamiento aceptable. Tres o cuatro conclusiones propias y pertinentes.	el razonamiento es superficial. Conclusiones genéricas o copiadas del material.	respuestas son irrelevantes. No escribe conclusiones o son irrelevantes.	
TOTAL					/16pts

Acciones a cumplir	Puntaje
Puntualidad y dedicación	1
Cumplimiento de tiempos establecidos	1
Conclusiones: ortografía y redacción	1
Puntaje total	

Comentarios respecto del desempeño del alumno

Nivel	Descripción
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo la mayoría de requerimientos.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad cumpliendo pocos requerimientos.
No aceptable	No demuestra entendimiento del problema o actividad.